pytest in Practice

Module 4 — Testing in Practice

Instructor **Angelo Sotgiu** · with **Battista Biggio**

Industrial Software Development M.Sc. in Computer Engineering, Cybersecurity and Artificial Intelligence University of Cagliari, Italy

At the end of Module 3, `heron` is packaged, formatted, linted and typed, but nothing checks what it **does**. Today it gets a test suite.

Concepts covered: **pytest** (discovery, `assert`, the report), **Arrange–Act–Assert**, **helpers and fixtures**, **parametrize**, the **test pyramid**, **test doubles** (stubs, fakes, mocks, `monkeypatch`), **coverage** and its limits, and a short **debugging toolkit**.

Where you work: lab-02-start

Module 4 is **Lab 2**, and it starts from branch `lab-02-start` of the course repository: the code at the end of Module 3 with some modifications (next three slides), but no tests. Create it in your clone and in your fork:

```
git fetch upstream
git switch -c lab-02-start upstream/lab-02-start   # your local branch
git push -u origin lab-02-start                    # the same branch in your fork
```

New in lab-02-start: findings (1/3)

`lab-02-start` is the code of Module 3 with **three changes** that I made for you. The first: each check now says **why** it adds points. A score alone does not explain a verdict; a **finding** does. `models.py` has a new data class:

```python
@dataclass(frozen=True)
class Finding:
    """One indicator that fired, and the points it adds to the score."""

    code: str
    detail: str
    weight: int
```

A finding records which indicator fired (`code`), the detail behind it (a link, a brand, the keywords found) and its `weight`, the points it adds.

New in lab-02-start: findings (2/3)

Each `*_score()` function becomes a **check** with a new name that returns a list of findings (empty when it does not fire): `check_urgency`, `check_links`, `check_brand`, `check_auth`, `check_reply_to`. For example:

```python
def check_auth(email: ParsedEmail) -> list[Finding]:
    """Return a finding when SPF or DMARC failed."""
    if "spf=fail" in email.auth_results or "dmarc=fail" in email.auth_results:
        return [Finding("auth_fail", "SPF or DMARC failed", W_AUTH_FAIL)]
    return []
```

`triage()` collects the findings of all the checks; the score is the sum of their weights, and `TriageResult` gets a fourth field, `findings`. The report prints them under each verdict, so you see why a message got its score:

```
prize-scam.eml                suspicious score=4
    - urgency (+2): verify, expires, act now, congratulations, winner, claim, gift card
    - reply_to_mismatch (+2): replies go to amaz0n-rewards.net
```

`ALL_CHECKS` now has the type `tuple[Callable[[ParsedEmail], list[Finding]], ...]`, and `--json` includes the findings.

New in lab-02-start: findings (3/3)

The findings show a problem: at the end of Module 3, `prize-scam.eml` (an advertising message) was *phishing* with score 9, because each of its seven marketing words added one point. A message full of advertising words scored like a forged one. So the second change counts the pressure words **once**: `keyword_score()` becomes `check_urgency()`, one finding of weight 2:

```python
def check_urgency(email: ParsedEmail) -> list[Finding]:
    """One finding, whatever the number of pressure keywords in the message."""
    hits = [kw for kw in KEYWORDS if kw in email.text]
    if not hits:
        return []
    return [Finding("urgency", ", ".join(hits), W_URGENCY)]
```

`prize-scam.eml` is now *suspicious*, score 4. The third change: the report ends with a summary line, `6 emails: 2 phishing, 1 suspicious, 3 benign` (`Counter`, from `collections`, counts the verdicts).

Each step: change, check, commit, push

Until now the safety net was the sorted output of `heron`, compared before and after each step. From step 2 on, the safety net is the **test suite** itself:

```
uv run pytest                                          # after each step: everything passes
git status                                             # check: only the files of the step
git add <the files of the step>
git commit -m "<message of the step>"
git push
```

- today we will only **add** tests
- the pre-commit hook of Module 3 runs ruff and mypy at every commit (mypy checks `src/` only)

Testing in pytest

Why pytest, not unittest

Python has two common test frameworks:

- **`unittest`** is in the standard library. Its style comes from Java's JUnit: each test is a method of a class that inherits from `unittest.TestCase`, and each check is a method such as `self.assertEqual(a, b)`, one method for each kind of comparison
- **pytest** is a separate package, and the de facto standard of the Python world. A test is a plain function, and a check is a plain `assert`

Why `heron` uses pytest:

- less code: no classes and no `assert*` methods to remember
- better failure reports: when an `assert` fails, pytest shows the values on both sides
- **fixtures** and **parametrize** (later today) remove repeated setup code and repeated tests
- many plugins, such as coverage; and pytest also runs old `unittest` tests, so a project can move to it gradually

Coming from unittest

unittest	**pytest**
subclass `unittest.TestCase`	a plain `test_*` function
`self.assertEqual(a, b)`	`assert a == b`
`self.assertRaises(E)`	`with pytest.raises(E):`
`setUp` / `tearDown` methods	fixtures

How pytest finds and runs tests

pytest discovers tests by name, with no base class and no registration:

- it collects files named `test_*.py` (under `testpaths`, such as `tests/`)
- inside them, functions named `test_*` (and methods `test_*` of classes named `Test*`)
- a plain `assert` is the whole assertion API: pytest rewrites it to print both sides on failure

```
uv run pytest                       # run everything
uv run pytest tests/test_scorer.py # one file
uv run pytest -k threshold          # only tests whose name contains "threshold"
uv run pytest -x                    # stop at the first failure
uv run pytest --co                  # collect only: list the tests, run nothing
```

There is no `pythonpath` setting: the tests import `heron` from the installed package, exactly as a user does.

Step 1: the test tools

Add the three test tools to the `dev` group (pytest, its coverage plugin, and Hypothesis):

```
uv add --dev pytest pytest-cov hypothesis   # pyproject.toml + uv.lock
```

Then configure pytest in `pyproject.toml`:

```
[tool.pytest.ini_options]
testpaths = ["tests"]
addopts = "-q"
```

`testpaths`: look for tests in `tests/` only. `addopts`: options added to every run; `-q` (quiet) gives a short output.

```
uv run pytest      # -> "no tests ran" (exit code 5: pytest found nothing to run)
git add pyproject.toml uv.lock
git commit -m "chore: add pytest, pytest-cov and hypothesis"
```

Step 2: the first test

Create the folder `tests/` with an empty `tests/__init__.py`, and `tests/test_scorer.py`:

```python
from heron.models import Verdict
from heron.scorer import PHISHING_THRESHOLD, SUSPICIOUS_THRESHOLD, verdict_for


def test_suspicious_threshold_is_inclusive() -> None:
    assert verdict_for(SUSPICIOUS_THRESHOLD) == Verdict.SUSPICIOUS
    assert verdict_for(SUSPICIOUS_THRESHOLD - 1) == Verdict.BENIGN


def test_phishing_threshold_is_inclusive() -> None:
    assert verdict_for(PHISHING_THRESHOLD) == Verdict.PHISHING
    assert verdict_for(PHISHING_THRESHOLD - 1) == Verdict.SUSPICIOUS
```

Each test checks a **boundary**: the score equal to the threshold, and one point below it. Boundaries are where bugs hide.

```
uv run pytest      # -> 2 passed
git add tests
git commit -m "test: the verdict thresholds are inclusive"
```

Watch it fail

A test you have never seen fail proves nothing. Break the code on purpose (do **not** commit this):

```
        if score >= PHISHING_THRESHOLD:
            return Verdict.PHISHING
-       if score >= SUSPICIOUS_THRESHOLD:
+       if score > SUSPICIOUS_THRESHOLD:
            return Verdict.SUSPICIOUS
```

```
>       assert verdict_for(SUSPICIOUS_THRESHOLD) == Verdict.SUSPICIOUS
E       AssertionError: assert <Verdict.BENIGN: 'benign'> == <Verdict.SUSP... 'suspicious'>
tests/test_scorer.py:6: AssertionError
1 failed, 1 passed
```

pytest prints the failing line, the values on both sides of the `assert`, and where it happened. Read all of it, then undo the change (`git restore src/heron/scorer.py`).

ruff and mypy did not notice: `>` is valid, well-typed code. Only a test that pins the behaviour sees the change.

Step 3: a builder for test messages

A test of a check needs a `ParsedEmail`. Writing all seven fields in every test hides what the test is about. Create `tests/helpers.py`:

```
"""Build test messages without writing .eml files."""

from heron.models import ParsedEmail


def make_email(
    *,
    sender: str = "Alice <alice@example.com>",
    from_domain: str | None = "example.com",
    reply_to_domain: str | None = None,
    auth_results: str = "spf=pass; dkim=pass; dmarc=pass",
    text: str = "hello, see you at the meeting.",
    links: tuple[str, ...] = (),
) -> ParsedEmail:
    """A harmless message; each keyword argument changes one field."""
    return ParsedEmail(
        email_id="test.eml",
        sender=sender,
        from_domain=from_domain,
        reply_to_domain=reply_to_domain,
        auth_results=auth_results,
        text=text,
        links=links,
    )
```

Step 3: defaults and keyword-only arguments

The **defaults** of `make_email` describe a **harmless** message: a normal sender, authentication that passed, no Reply-To, no links, no pressure words. A test changes only the field it is about.

The `*` at the start of the parameters makes every argument **keyword-only**: a call must name each argument it passes. So a test reads `make_email(reply_to_domain="mail-verify.net")`, and the reader sees at once the one field that matters.

```
git add tests/helpers.py
git commit -m "test: build test messages with make_email"
```

Arrange, Act, Assert

A readable test has three parts, in this order, separated by a blank line:

```python
def test_urgency_fires_once_for_many_keywords() -> None:
    email = make_email(text="urgent: verify your password immediately")

    findings = check_urgency(email)

    assert [f.code for f in findings] == ["urgency"]
    assert findings[0].weight == W_URGENCY
```

- **Arrange** builds the input; **Act** is one call; **Assert** checks what came out
- two *Act* steps in one test usually mean two tests; a long *Arrange* means a helper is missing

Step 4: fires, and does not fire

Every check gets two kinds of test: it **fires** on the input it is about, and it stays **silent** otherwise. `tests/test_features.py` continues:

```python
def test_urgency_is_silent_without_keywords() -> None:
    assert check_urgency(make_email()) == []


def test_reply_to_fires_for_another_domain() -> None:
    email = make_email(reply_to_domain="mail-verify.net")

    findings = check_reply_to(email)

    assert [(f.code, f.weight) for f in findings] == [("reply_to_mismatch", W_REPLY_TO_MISMATCH)]


def test_reply_to_is_silent_when_a_domain_is_missing() -> None:
    assert check_reply_to(make_email(reply_to_domain=None)) == []
    assert check_reply_to(make_email(from_domain=None, reply_to_domain="x.example")) == []
```

Half of the bugs of a detector live in the **false positives**: the silent cases matter as much as the firing ones.

Step 4: a harmless message scores zero

One more test, in `tests/test_scorer.py`, checks the whole engine on the harmless default message:

```python
def test_a_harmless_message_scores_zero() -> None:
    result = triage(make_email())
    assert result.score == 0
    assert result.verdict == Verdict.BENIGN
    assert result.findings == ()
```

It also checks the builder: if a default of `make_email` fired a check, every other test would start from a wrong message.

```
uv run pytest      # -> 7 passed
git add tests
git commit -m "test: one test per check, when it fires and when it does not"
```

Step 5: parametrize

The link check has several interesting inputs. `@pytest.mark.parametrize` runs one test body once per row, and each row is a separate test in the report:

```python
@pytest.mark.parametrize(
    ("links", "codes"),
    [
        (("http://192.0.2.44/login",), ["ip_link"]),
        (("https://xn--pypal-4ve.com/signin",), ["punycode_link"]),
        (("https://www.example.com/news",), []),
        (("http://192.0.2.1/a", "http://192.0.2.2/b"), ["ip_link", "ip_link"]),
    ],
    ids=["bare-ip", "punycode", "plain", "two-ip-links"],
)
def test_check_links(links: tuple[str, ...], codes: list[str]) -> None:
    assert [f.code for f in check_links(make_email(links=links))] == codes
```

`ids=` gives each row a readable name in the report (`test_check_links[punycode]`), so a failure points straight at the case. The last row pins a behaviour: two IP links give **two** findings.

`git commit -m "test: parametrize the link check"` (11 passed)

Stacking parametrize

Two `parametrize` marks on one test run the **cross-product** of their rows (a generic example):

```python
@pytest.mark.parametrize("auth", [AUTH_PASS, AUTH_FAIL])
@pytest.mark.parametrize("has_link", [True, False])
def test_matrix(auth, has_link): ...        # 2 × 2 = 4 test cases
```

Use it for an input matrix; keep the tables small enough to read.

Step 6: fixtures

A **fixture** is a function that prepares something a test needs. pytest finds fixtures in `tests/conftest.py` by themselves; create it:

```
import pytest

from heron.models import ParsedEmail
from tests.helpers import make_email


@pytest.fixture
def benign_email() -> ParsedEmail:
    return make_email()


@pytest.fixture
def phishing_email() -> ParsedEmail:
    return make_email(
        sender="PayPal Support <security@paypa1-secure.net>",
        from_domain="paypa1-secure.net",
        reply_to_domain="mail-verify.net",
        auth_results="spf=fail; dmarc=fail",
        text="urgent: your account is suspended.",
        links=("http://192.0.2.44/login",),
    )
```

Step 6: inject a fixture by name

A test does not call a fixture. It **names** it as a parameter, and pytest runs the fixture and passes in the result (`tests/test_scorer.py`):

```
def test_fixtures_are_injected_by_name(
    benign_email: ParsedEmail, phishing_email: ParsedEmail
) -> None:
    assert triage(benign_email).verdict == Verdict.BENIGN
    assert triage(phishing_email).verdict == Verdict.PHISHING
```

The parameter name must match the fixture name exactly. Each test gets a **fresh** result: one test cannot change the message another test sees.

`git commit -m "test: shared fixtures in conftest"` (12 passed)

helpers.py or conftest.py?

The two files have different jobs:

File	Holds	How tests use it
`tests/helpers.py`	plain functions (`make_email`)	a normal `import`
`tests/conftest.py`	**fixtures** (and pytest hooks)	by name, as a parameter; **never import it**

A builder with arguments belongs in `helpers.py`: each test calls it with its own fields. A ready-made object that many tests share belongs in a fixture.

Step 7: the CLI end to end

The unit tests check the parts. One test drives the whole program through its public entry point, `main()`, on the six samples (`tests/test_cli.py`):

```python
from pathlib import Path

import pytest

from heron.cli import main

SAMPLES = Path(__file__).parent.parent / "data" / "samples"


def test_cli_triages_the_six_samples(capsys: pytest.CaptureFixture[str]) -> None:
    exit_code = main([str(SAMPLES)])

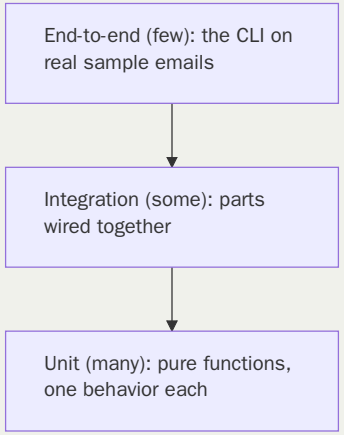
    out = capsys.readouterr().out
    assert exit_code == 0
    assert out.splitlines()[-1] == "6 emails: 2 phishing, 1 suspicious, 3 benign"
```

`capsys` is a built-in fixture that captures what the program prints. `main()` takes its arguments as a list, so the test needs no shell.

`git commit -m "test: the CLI end to end on the samples"` (13 passed)

The test pyramid

You now have all three layers. Keep their proportions in this shape:



Unit tests are many, fast, and point at the broken function. **End-to-end** tests are few and slower, but prove that the parts work together.

Test doubles

The dependency problem

The pure checks were easy to test. Code at the edges touches the world, and a test must control it:

The edge	**In `heron`**	**How a test controls it**
files	`cli` reads `.eml` files	`tmp_path`: a fresh temporary folder per test
output, exit code	`main(argv)` prints and returns	`capsys` and the return value
the checks	`scorer` loops over `ALL_CHECKS`	replace the tuple with `monkeypatch`
time, network	not yet: a future check might need them	pass them in as **parameters**

The way out is always the same: replace the collaborator with a controlled substitute, a **test double**, and test your logic in isolation.

The cast of doubles

Test double is the umbrella term (like a stunt double in a film). The kinds differ in what you check:

- **Stub**: returns canned answers; you check the result of the code under test
- **Fake**: a working, lightweight implementation (a dict instead of a WHOIS service or a database); you check the result
- **Mock**: records the calls it received; you check the interaction ("was the lookup called with `fresh.example`?")
- **Spy**: the real thing, wrapped to record calls

Rule of thumb: check results when you can, because that survives refactoring. Check interactions only when the call itself is the behaviour.

Step 8: monkeypatch

pytest's built-in `monkeypatch` fixture replaces an attribute for one test and undoes the change afterwards. Here it replaces all the checks with one **stub** (`tests/test_doubles.py`):

```python
import pytest

from heron import scorer
from heron.models import Finding, ParsedEmail, Verdict
from tests.helpers import make_email


def always_flag(email: ParsedEmail) -> list[Finding]:
    """A stub check: it fires on every message."""
    return [Finding("stub", "always fires", 7)]


def test_monkeypatch_replaces_the_checks(monkeypatch: pytest.MonkeyPatch) -> None:
    # scorer.py imported ALL_CHECKS by name: patch it where it is used
    monkeypatch.setattr(scorer, "ALL_CHECKS", (always_flag,))

    result = scorer.triage(make_email())

    assert result.verdict == Verdict.PHISHING
    assert [f.code for f in result.findings] == ["stub"]
```

`git commit -m "test: replace the checks with monkeypatch"` (14 passed)

Patch where the name is used

In Python, a **name** is a reference to an object. Importing something from a module does not copy the object: it creates a new name, in the module that imports, which points to the same object.

`ALL_CHECKS` is defined in `heron.features` and imported and used in `heron.scorer`. When `triage()` runs, it looks up the name `ALL_CHECKS` in **its own module**, `scorer`. It never reads `features.ALL_CHECKS`.

`monkeypatch.setattr(module, "NAME", value)` moves **one** name to a new object, for one test only. The other name, and the tuple itself, do not change. So we patch the name **where it is used** (`heron.scorer`), not where it is defined (`heron.features`).

`monkeypatch.setenv` and `delenv` change environment variables in the same way;
`unittest.mock.patch("heron.scorer.ALL_CHECKS", ...)` does the same job as `setattr`, with the same rule.

Mocks

A **mock** is a double that **records** every call it receives. Python has one in the standard library: `unittest.mock.Mock`. Use it when the call itself is the behaviour, for example an alert that must be sent (an example, not code of `heron`):

```python
def alert_if_phishing(result: TriageResult, send: Callable[[str], None]) -> None:
    """Send an alert to the security team for each phishing verdict."""
    if result.verdict == Verdict.PHISHING:
        send(f"phishing: {result.email_id}")


def test_a_phishing_verdict_sends_one_alert() -> None:
    send = Mock()  # stands in for the real e-mail or chat service
    alert_if_phishing(TriageResult("a.eml", 9, Verdict.PHISHING, ()), send)
    send.assert_called_once_with("phishing: a.eml")
```

The `Mock` accepts any call and remembers it; `assert_called_once_with` checks that the alert was sent once, with the right text. The test sends no real message.

The function here returns nothing, but you can set custom return values and additional methods/attributes. A mock encodes your **beliefs** about the real service: if they are wrong, the tests pass and the program fails.

Coverage

Step 9: measure coverage, with a gate

Coverage measures which lines (and which branches of each `if`) the tests ran. Configure it in `pyproject.toml`:

```toml
[tool.coverage.run]
source = ["heron"]
branch = true

[tool.coverage.report]
show_missing = true
fail_under = 85
```

`branch = true` counts an `if` as covered only if the tests took **both** ways. `fail_under = 85` makes `pytest --cov` fail below 85%.

Add `.hypothesis/` and `mutants/` to `.gitignore`, then:

```
uv run pytest --cov
git add pyproject.toml .gitignore
git commit -m "chore: measure branch coverage with a gate"
```

Read the report, not the number

An example report:

```
Name                       Stmts   Miss Branch BrPart  Cover   Missing
----------------------------------------------------------------------
src/heron/__init__.py          1      0      0      0   100%
src/heron/__main__.py          2      2      0      0     0%   3-5
src/heron/cli.py              40      4      8      1    90%   52-54, 56
src/heron/features.py         38      0     12      0   100%
src/heron/models.py           26      0      0      0   100%
src/heron/parser.py           14      0      2      0   100%
src/heron/scorer.py           16      0      6      0   100%
----------------------------------------------------------------------
TOTAL                        137      6     28      1    96%
```

Look at the **Missing** column, not at 96%:

- `cli.py` 52–54 is the error for a missing path, and 56 is `--json`: both are behaviour a user relies on.
- `__main__.py` only calls `main()` for `python -m heron`: noise, not worth a test
- for each line you decide to cover, say in one sentence what breaks for a user if it is wrong

Coverage does not certify behaviour

Coverage counts which lines **ran**, not which results were **checked**. Try this change in `check_brand` (do not commit it):

```
      return [
          Finding("brand_mismatch", brand, W_BRAND_MISMATCH)
          for brand in BRANDS
-         if brand in low and brand + ".com" not in low
+         if brand in low
      ]
```

All 14 tests still pass, `features.py` stays at 100%, and the samples still give 2 / 1 / 3. Yet `heron` now flags **PayPal's own mail** (`service@paypal.com`) as brand impersonation. The line ran; no test checked the case of the real domain.

Coverage finds **forgotten** code. It never proves that the code you ran is correct.

A silent regression

The `git bisect` demo found the commit `refactor: compare brands with the sender domain only`. In today's code the same change is one line of `check_brand` (do not commit it):

```
-     low = email.sender.lower()
+     low = email.from_domain or ""
```

In the bisect repository, with no tests, the change was silent: `password-reset-urgent.eml` lost its 3 brand points, nothing failed, and we needed `git bisect` to find the commit. Make the change on today's code and run the suite: the end-to-end test of step 7 fails at once, because the last line becomes `6 emails: 1 phishing, 2 suspicious, 3 benign`. Then undo it (`git restore src/heron/features.py`).

A regression is caught only by a test that **pins the behaviour**, and only if someone **runs** it.

Debugging

Debugging toolkit: read, then stop

When a test fails, the **traceback** is your first tool. Read it from the **bottom up**: the last line is the error, and the frame just above it is where it happened.

To look inside a running program, stop it:

```
def check_brand(email: ParsedEmail) -> list[Finding]:
    breakpoint()          # Python's built-in: opens the pdb debugger here
    ...
```

```
uv run pytest -x --pdb          # open pdb at the first failure, with all local variables
uv run pytest --lf              # re-run only the tests that failed last time
```

In pdb: `p name` prints a value, `n` runs the next line, `s` steps into a call, `c` continues, `q` quits. IDE breakpoints (VS Code, PyCharm) do the same with the mouse. Remove every `breakpoint()` before you commit (ruff rule `T100` can enforce it).

Demo: pdb inside check_brand

Why does `password-reset-urgent.eml` get a `brand_mismatch` finding, although its domain contains no brand name? I stop the code and look:

```
# 1. add `breakpoint()` as the first line of check_brand in src/heron/features.py
uv run heron data/samples/password-reset-urgent.eml
(Pdb) p email.sender        # 'Microsoft 365 IT Helpdesk <it-helpdesk@xn--microsft-w2c.com>'
(Pdb) p email.from_domain   # 'xn--microsft-w2c.com'
(Pdb) n                     # run the next line: low = ...
(Pdb) p low                 # the whole From header, in lower case
(Pdb) c                     # let the run finish
```

Which part of the From header carries the brand, and why does the bisect regression ("compare brands with the sender domain only") lose it? After debugging, remember to **remove the `breakpoint()`** and check that `git diff` is empty.

Debugging toolkit: measure, then search

Profile before you optimise. `cProfile` (standard library) counts the calls and the time of each function:

```
uv run python -m cProfile -s tottime -m heron data/samples | grep -A 8 ncalls
```

On six emails, the top lines are Python **loading modules** (`marshal.loads`, `exec`), not `heron`'s checks: the program spends most of its time starting up. Guessing the slow part is usually wrong; measuring takes seconds.

Takeaways

- pytest finds `test_*` functions and explains a failing `assert`; a test you have never seen fail proves nothing
- Arrange–Act–Assert; a builder (`make_email`) for the arrange step, fixtures for shared objects, `parametrize` for tables of cases; most tests at the bottom of the pyramid
- test every check both ways: it fires, and it stays silent
- isolate logic from the world with doubles: stubs and fakes for results, mocks for interactions; patch where the name is **used**
- coverage finds forgotten code; it does not certify results. A regression is caught only by a test that pins the behaviour, and only if it runs

Appendix: every step in full

How to read the appendix

The slides of the lecture show only the key lines of each step. The next slides show the **complete change** of every step, as `git diff` prints it, so that you can check your code line by line, or type a step you missed. The same commits are on the branch `lab-02-solution` of `unica-isde/heron`.

- a line that starts with `-` is removed, a line that starts with `+` is added, a line that starts with a space is unchanged and only shows where the change goes
- `@@ -12,3 +12,3 @@` gives the position of the lines that follow: line 12 of the old file (3 lines shown) becomes line 12 of the new file
- a **new file** is shown in full, as plain code; you write it **from scratch**, unless the slide says which file to copy it from
- when a step needs more than one slide, the change continues on the next slide, in order

Step 1 in full

Commit `chore: add pytest, pytest-cov and hypothesis`: the complete change to `pyproject.toml`. `uv add --dev pytest pytest-cov hypothesis` writes the first hunk and updates `uv.lock` (do not type `uv.lock`); you type the `[tool.pytest.ini_options]` table.

```
@@ -15,2 +15,3 @@
    "ruff", "mypy", "pre-commit", "sphinx", "sphinx-rtd-theme",
+   "pytest", "pytest-cov", "hypothesis",
  ]
@@ -37 +38,5 @@
  files = ["src"]
+
+[tool.pytest.ini_options]
+testpaths = ["tests"]
+addopts = "-q"
```

Step 2 in full

Commit `test: the verdict thresholds are inclusive`: two new files. `tests/__init__.py` is **empty** (it makes `tests` a package, so `from tests.helpers import …` works from step 3 on).

New file `tests/test_scorer.py`, written from scratch:

```python
from heron.models import Verdict
from heron.scorer import PHISHING_THRESHOLD, SUSPICIOUS_THRESHOLD, verdict_for


def test_suspicious_threshold_is_inclusive() -> None:
    assert verdict_for(SUSPICIOUS_THRESHOLD) == Verdict.SUSPICIOUS
    assert verdict_for(SUSPICIOUS_THRESHOLD - 1) == Verdict.BENIGN


def test_phishing_threshold_is_inclusive() -> None:
    assert verdict_for(PHISHING_THRESHOLD) == Verdict.PHISHING
    assert verdict_for(PHISHING_THRESHOLD - 1) == Verdict.SUSPICIOUS
```

Step 3 in full

Commit `test: build test messages with make_email`: one new file.

New file `tests/helpers.py`, written from scratch:

```
"""Build test messages without writing .eml files."""

from heron.models import ParsedEmail


def make_email(
    *,
    sender: str = "Alice <alice@example.com>",
    from_domain: str | None = "example.com",
    reply_to_domain: str | None = None,
    auth_results: str = "spf=pass; dkim=pass; dmarc=pass",
    text: str = "hello, see you at the meeting.",
    links: tuple[str, ...] = (),
) -> ParsedEmail:
    """A harmless message; each keyword argument changes one field."""
    return ParsedEmail(
        email_id="test.eml",
        sender=sender,
        from_domain=from_domain,
        reply_to_domain=reply_to_domain,
        auth_results=auth_results,
        text=text,
        links=links,
    )
```

Step 4 in full (1/3)

Commit `test: one test per check, when it fires and when it does not`: the complete change.

New file `tests/test_features.py`, written from scratch:

```python
from heron.features import W_REPLY_TO_MISMATCH, W_URGENCY, check_reply_to, check_urgency
from tests.helpers import make_email


def test_urgency_fires_once_for_many_keywords() -> None:
    email = make_email(text="urgent: verify your password immediately")

    findings = check_urgency(email)

    assert [f.code for f in findings] == ["urgency"]
    assert findings[0].weight == W_URGENCY


def test_urgency_is_silent_without_keywords() -> None:
    assert check_urgency(make_email()) == []
```

Step 4 in full (2/3)

Step 4, continued: the rest of `tests/test_features.py`.

```
def test_reply_to_fires_for_another_domain() -> None:
    email = make_email(reply_to_domain="mail-verify.net")

    findings = check_reply_to(email)

    assert [(f.code, f.weight) for f in findings] == [("reply_to_mismatch", W_REPLY_TO_MISMATCH)]


def test_reply_to_is_silent_when_a_domain_is_missing() -> None:
    assert check_reply_to(make_email(reply_to_domain=None)) == []
    assert check_reply_to(make_email(from_domain=None, reply_to_domain="x.example")) == []
```

Step 4 in full (3/3)

Step 4, continued: the change to `tests/test_scorer.py`. Add both files before the commit.

```
@@ -1,3 +1,4 @@
 from heron.models import Verdict
-from heron.scorer import PHISHING_THRESHOLD, SUSPICIOUS_THRESHOLD, verdict_for
+from heron.scorer import PHISHING_THRESHOLD, SUSPICIOUS_THRESHOLD, triage, verdict_for
+from tests.helpers import make_email

@@ -12 +13,8 @@
     assert verdict_for(PHISHING_THRESHOLD - 1) == Verdict.SUSPICIOUS
+
+
+def test_a_harmless_message_scores_zero() -> None:
+    result = triage(make_email())
+    assert result.score == 0
+    assert result.verdict == Verdict.BENIGN
+    assert result.findings == ()
```

Step 5 in full (1/2)

Commit `test: parametrize the link check`: the complete change to `tests/test_features.py`.

```
@@ -1,2 +1,10 @@
-from heron.features import W_REPLY_TO_MISMATCH, W_URGENCY, check_reply_to, check_urgency
+import pytest
+
+from heron.features import (
+    W_REPLY_TO_MISMATCH,
+    W_URGENCY,
+    check_links,
+    check_reply_to,
+    check_urgency,
+)
 from tests.helpers import make_email
```

Step 5 in full (2/2)

Step 5, continued: the end of `tests/test_features.py`.

```
@@ -28 +36,15 @@
     assert check_reply_to(make_email(from_domain=None, reply_to_domain="x.example")) == []
+
+
+@pytest.mark.parametrize(
+    ("links", "codes"),
+    [
+        (("http://192.0.2.44/login",), ["ip_link"]),
+        (("https://xn--pypal-4ve.com/signin",), ["punycode_link"]),
+        (("https://www.example.com/news",), []),
+        (("http://192.0.2.1/a", "http://192.0.2.2/b"), ["ip_link", "ip_link"]),
+    ],
+    ids=["bare-ip", "punycode", "plain", "two-ip-links"],
+)
+def test_check_links(links: tuple[str, ...], codes: list[str]) -> None:
+    assert [f.code for f in check_links(make_email(links=links))] == codes
```

Step 6 in full (1/2)

Commit `test: shared fixtures in conftest`: the complete change.

New file `tests/conftest.py`, written from scratch:

```
import pytest

from heron.models import ParsedEmail
from tests.helpers import make_email


@pytest.fixture
def benign_email() -> ParsedEmail:
    return make_email()


@pytest.fixture
def phishing_email() -> ParsedEmail:
    return make_email(
        sender="PayPal Support <security@paypa1-secure.net>",
        from_domain="paypa1-secure.net",
        reply_to_domain="mail-verify.net",
        auth_results="spf=fail; dmarc=fail",
        text="urgent: your account is suspended.",
        links=("http://192.0.2.44/login",),
    )
```

Step 6 in full (2/2)

Step 6, continued: the change to `tests/test_scorer.py`. Add both files before the commit.

```
@@ -1,2 +1,2 @@
-from heron.models import Verdict
+from heron.models import ParsedEmail, Verdict
 from heron.scorer import PHISHING_THRESHOLD, SUSPICIOUS_THRESHOLD, triage, verdict_for
@@ -20 +20,8 @@
     assert result.findings == ()
+
+
+def test_fixtures_are_injected_by_name(
+    benign_email: ParsedEmail, phishing_email: ParsedEmail
+) -> None:
+    assert triage(benign_email).verdict == Verdict.BENIGN
+    assert triage(phishing_email).verdict == Verdict.PHISHING
```

Step 7 in full

Commit `test: the CLI end to end on the samples`: one new file.

New file `tests/test_cli.py`, written from scratch:

```python
from pathlib import Path

import pytest

from heron.cli import main

SAMPLES = Path(__file__).parent.parent / "data" / "samples"


def test_cli_triages_the_six_samples(capsys: pytest.CaptureFixture[str]) -> None:
    exit_code = main([str(SAMPLES)])

    out = capsys.readouterr().out
    assert exit_code == 0
    assert out.splitlines()[-1] == "6 emails: 2 phishing, 1 suspicious, 3 benign"
```

Step 8 in full

Commit `test: replace the checks with monkeypatch`: one new file.

New file `tests/test_doubles.py`, written from scratch:

```python
import pytest

from heron import scorer
from heron.models import Finding, ParsedEmail, Verdict
from tests.helpers import make_email


def always_flag(email: ParsedEmail) -> list[Finding]:
    """A stub check: it fires on every message."""
    return [Finding("stub", "always fires", 7)]


def test_monkeypatch_replaces_the_checks(monkeypatch: pytest.MonkeyPatch) -> None:
    # scorer.py imported ALL_CHECKS by name: patch it where it is used
    monkeypatch.setattr(scorer, "ALL_CHECKS", (always_flag,))

    result = scorer.triage(make_email())

    assert result.verdict == Verdict.PHISHING
    assert [f.code for f in result.findings] == ["stub"]
```

Step 9 in full

Commit `chore: measure branch coverage with a gate`: the complete change.

`.gitignore`:

```
@@ -14 +14,3 @@
 .env
+.hypothesis/
+mutants/
```

`pyproject.toml`:

```
@@ -42 +42,9 @@
 addopts = "-q"
+
+[tool.coverage.run]
+source = ["heron"]
+branch = true
+
+[tool.coverage.report]
+show_missing = true
+fail_under = 85
```